

How to not write parsers

Wow, that's a lot of grammars, too bad I'm not reading them

Curtis Chin Jen Sem



Scan for these slides

It started with wanting `.cabal` files to look nice

- syntax-highlighting is nice.
- started with wanting it for `.cabal` files in my editor helix.
 - pre-existing `.cabal` tree-sitter grammar by [Magnus Therning](#)

It started with wanting `.cabal` files to look nice

- syntax-highlighting is nice.
- started with wanting it for `.cabal` files in my editor helix.
 - pre-existing `.cabal` tree-sitter grammar by Magnus Therning



It started with wanting `.cabal` files to look nice

- syntax-highlighting is nice.
- started with wanting it for `.cabal` files in my editor helix.
 - pre-existing `.cabal` tree-sitter grammar by Magnus Therning



- a PR by Ananda Umamil fixed the segfaults
- but then I wanted `cabal.project`, which look like `.cabal` files
- Grammar?

Is this a valid `.cabal` file?

```
Library {  
    build-depends: base ≥ 4.0 && < 4.2,  
                  syb  ≥ 0.1 && < 0.2  
  
    if impl(ghc < 6.9) {  
        buildable: False  
    }  
}
```

Is this a valid `.cabal` file?

```
Library {  
    build-depends: base ≥ 4.0 && < 4.2,  
                  syb  ≥ 0.1 && < 0.2  
  
    if impl(ghc < 6.9) {  
        buildable: False  
    }  
}
```

`cabal` says **valid, no warnings**

- lots of legacy syntax
- there is a whole **brace-delimited** syntax

How about this one?

```
library
  build-depends: base

-- Duplicating inclusion plugin conditions
if flag(eval)
  cpp-options: -Dhls_eval
```

How about this one?

```
library
  build-depends: base

-- Duplicating inclusion plugin conditions
  if flag(eval)
    cpp-options: -Dhls_eval
```

cabal says **valid, no warnings**

- a comment at **column zero** does not end the stanza
- the **if** still belongs to the **library** above it
- in `haskell-language-server.cabal`

And these? Which of these is right?

library

```
if os(windows)
    build-depends: Win32
elif os(linux)
    build-depends: unix
```

library

```
if os(windows)
    build-depends: Win32
elseif os(linux)
    build-depends: unix
```

And these? Which of these is right?

library

```
if os(windows)
    build-depends: Win32
elif os(linux)
    build-depends: unix
```

cabal **valid, since cabal-version: 2.2**

library

```
if os(windows)
    build-depends: Win32
elseif os(linux)
    build-depends: unix
```

cabal ***invalid subsection, ignored***

- `elif` is the real one.
- `elseif` still **parses**, warns, and drops the branch
- both successfully `cabal build`!

That's a lot of legacy

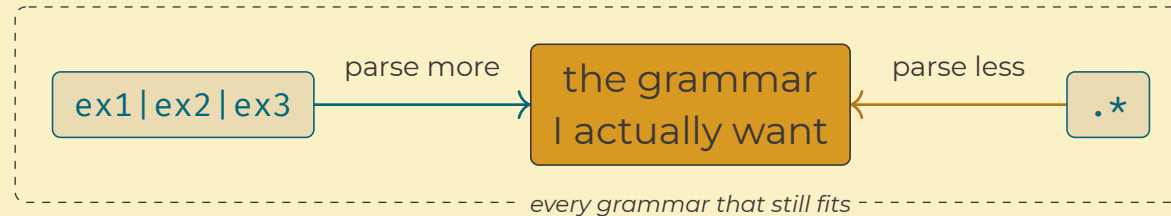
- twenty years of accumulated legacy
- writing parsers is already hard enough
- coverage is a problem

That's a lot of legacy

- twenty years of accumulated legacy
- writing parsers is already hard enough
- coverage is a problem
- so what if I don't?
- instead of a parsing problem,

That's a lot of legacy

- twenty years of accumulated legacy
- writing parsers is already hard enough
- coverage is a problem
- so what if I don't?
- instead of a parsing problem,
- ... how about tackling it like a constraint solving problem?



Tree-sitter

- a parser generator DSL for **incremental** parsers
- grammar in javascript, generates parsers in C
- out comes a **concrete** syntax tree

```
library
  build-depends: base ≥ 4.9

(library
  type: (section_type)
  properties: (property_or_conditional_block
    (field
      name: (field_name)
      value: (field_value
        (identifier)
        (constraint_op)
        (version))))))
```

Tree-sitter

- **always** returns a tree even if syntax is invalid

```
library
```

```
  build-depends base ≥ 4.9
```

↑ no colon

```
(library
  type: (section_type))
(ERROR
  (field_name)
  (identifier)
  (constraint_op)
  (version))
```

Tree-sitter

- **always** returns a tree even if syntax is invalid

```
library
  build-depends base ≥ 4.9
                ↑ no colon
```

```
(library
  type: (section_type))
(ERROR
  (field_name)
  (identifier)
  (constraint_op)
  (version))
```

- most common used for syntax highlighting
- editors have to highlight **actively** edited files
- also used for tags, tree-based navigation, code-folding

Pattern matching parse trees

```
library core
  build-depends: base
```

```
its parse tree
(cabal
 (sections
  (library
   type: (section_type)
   name: (section_name)
   properties: (property_or_conditional_block
    (field
     name: (field_name)
     value: (field_value
      (identifier)))))))))
```

query

```
(library
  name: (section_name) @name)
```

Pattern matching parse trees

```
library core  
  build-depends: base
```

```
its parse tree  
(cabal  
  (sections  
    (library  
      type: (section_type)  
      name: (section_name)  
      properties: (property_or_conditional_block  
        (field  
          name: (field_name)  
          value: (field_value  
            (identifier))))))
```

query

```
(library  
  name: (section_name) @name)
```

@name captures **core**

- change the capture to `type: (section_type)` and you get `library`

Using captures

- going from captures to behavior

```
pkg.cabal, three stanzas:  
library core  
    build-depends: base  
executable mypkg  
    ghc-options: -O2  
flag fast  
    default: False
```

```
tags.scm, one pattern per symbol kind  
(library      ...  @definition.module  
(executable   ...  @definition.function  
(flag         ...  @definition.constant  
... is  name: (section_name) @name)
```

what a tags reader gets

```
→ core  module  
→ mypkg function  
→ fast  constant
```

- the outer node decides which stanza matches
- @name gives the symbol's text, definition.<kind> gives its kind
- so tags.scm turns node names into definition variants

Features as queries

- pull nodes out of the tree, tag each one with a capture

query file	pulls out of the tree	which Helix exposes as	lines
<code>highlights.scm</code>	every literal, name, operator	colour	101
<code>textobjects.scm</code>	sections and their bodies	select or jump a stanza	45
<code>indents.scm</code>	section and <code>if</code> bodies	where the cursor goes on enter	32
<code>tags.scm</code>	section names	the symbol picker	15
<code>locals.scm</code>	<code>flag</code> defs and their uses	go-to-definition on a flag	11
<code>rainbows.scm</code>	parens in conditions	bracket pair colours	7

- query one single tree six different ways easily
- only have to give **named nodes**

Features as queries

- pull nodes out of the tree, tag each one with a capture

query file	pulls out of the tree	which Helix exposes as	lines
<code>highlights.scm</code>	every literal, name, operator	colour	101
<code>textobjects.scm</code>	sections and their bodies	select or jump a stanza	45
<code>indents.scm</code>	section and <code>if</code> bodies	where the cursor goes on enter	32
<code>tags.scm</code>	section names	the symbol picker	15
<code>locals.scm</code>	<code>flag</code> defs and their uses	go-to-definition on a flag	11
<code>rainbows.scm</code>	parens in conditions	bracket pair colours	7

- query one single tree six different ways easily
- only have to give **named nodes**

grammar and queries define the solution space

Can the queries even compile?

- every pattern in every query file
- resolved against the generated parser

the pattern	what check-queries says
<code>(library name: (section_name) @name)</code>	captures core
<code>(librari name: (section_name) @name)</code>	Invalid node type "librari"
<code>(library nmae: (section_name) @name)</code>	Invalid field name "nmae"

- node types and field names are checked
- the grammar is now forced to support every query

Harvesting corpora

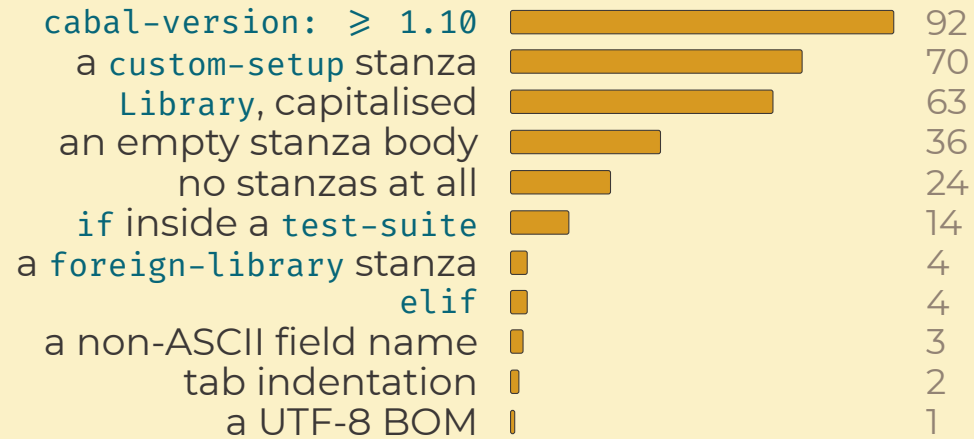
- no formal **cabal grammar**
- ...**but**, there's an elaborate testsuite
- **find** over two testsuites, cabal and HLS

harvested from	files
cabal-testsuite/PackageTests	760
Cabal-tests/.../ParserTests	84
haskell-language-server/test/testdata	64
cabal-install/tests	35
the two projects' .cabal files	24

- **943/967** test-cases, median **14 lines**

Does every real file parse?

- `parse-corpus` reads all 967 harvested files
- asserts zero `ERROR`, zero `MISSING`
- `tree-sitter-cabal` failed 237



What did I capture?

- `extract-golden` records `capture`, `span` and `text` over fixtures
- and diffs that against a committed set

	library	core
the capture I asked for	<code>name</code>	<code>3,8-3,12</code>
one node to the left	<code>name</code>	<code>3,0-3,7</code>
		<code>core</code>
		<code>library</code>

- the capture slid to the `wrong node`
- span `and` text both wrong

What actually reaches the screen?

- `highlight-golden` records the `winning` capture
- one row per token, from rendered output

the same `.`, in two fields

```
hs-source-dirs: .   ln 26  string.special.path
description: ... .  ln  8  string
```

What actually reaches the screen?

- `highlight-golden` records the `winning` capture
- one row per token, from rendered output

the same `.`, in two fields

```
hs-source-dirs: .   ln 26  string.special.path
description: ... .  ln  8  string
```

- one node type interpreted two `different` ways
- swap those patterns in the file
- and the result changes

parse never fails

```
readFields :: ByteString → Either ParseError [Field]    -- Cabal
parse      :: Text       → Tree                        -- tree-sitter
```

```
library
  build-depends base ≥ 4.9
                        ↑ no colon
```

*parses
to →*

```
(library
  type: (section_type))
(ERROR
  (field_name)
  (identifier)
  (constraint_op)
  (version))
```

did it parse? always **yes**

parse-corpus constrains

match never chooses

```
match :: Query → Tree → [Match]
```

```
hs-source-dirs:  • → string  
                 → string.special.path
```

match returns everything

extract-golden and highlight-golden constrain

What's my variable?

each constraint is a fold

constraints $C_1, \dots, C_n$, each form a fold from **Tree** into some M_i ,
with expected values $e_i \in M_i$ over fixtures X

$$C_i(\text{parse}_{\mathbf{g}} x) = e_i(x), \quad \forall x \in X$$

solve for **g**, the grammar and the queries

- the constraints are the spec
- every entry is a sample
- the grammar is whatever fits

Tree constraints can be phrased as **foldMap**

`foldMap` :: `Monoid m` $\Rightarrow$ `(a  $\rightarrow$  m)  $\rightarrow$  [a]  $\rightarrow$  m`

constrain queries by grammar

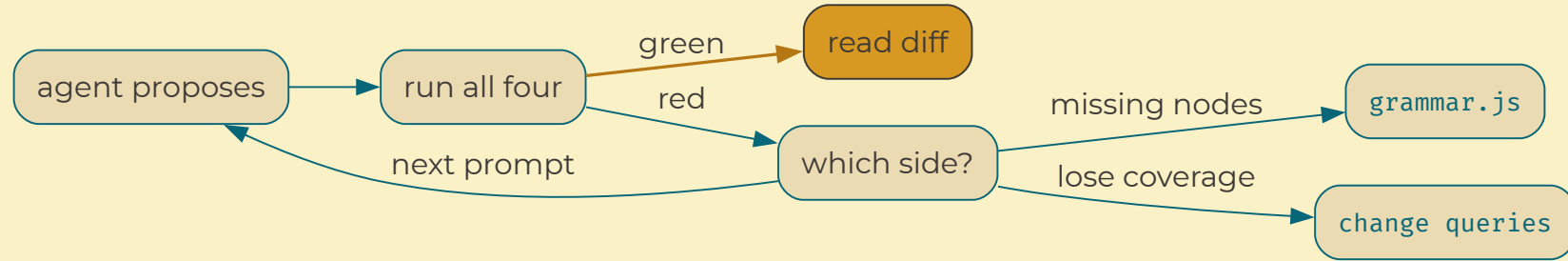
C_1 `check-queries` `queries` $\subseteq$ `grammar` assertion

constrain `g` over files

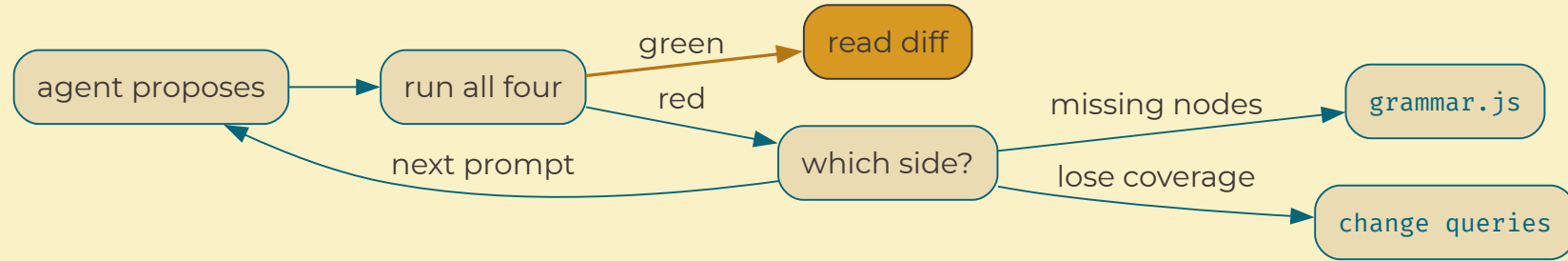
C_2	<code>parse-corpus</code>	<code>foldMap</code> <code>all</code>	<code>. parse</code>	assertion
C_3	<code>extract-golden</code>	<code>foldMap</code> <code>id</code>	<code>. match . parse</code>	<code>value.path</code> 25,18-25,19 <code>'.'</code> per capture
C_4	<code>highlight-golden</code>	<code>foldMap</code> <code>last</code>	<code>. match . parse</code>	26 <code>string.special.path</code> <code>'.'</code> per token

turn these `monoid` values into tests by **rendering**

Both paths are the same job for the agent



Both paths are the same job for the agent



measure, edit, re-run

- only the **grammar and queries** change
- manual step when everything is correct
- check **is this** the change I wanted?

Constraints require different sets



I didn't write the parser
just read everything else

the agent wrote **from**
7 grammars, 4 C scanners 4 constraints, and 2 corpora

967 cabal files, zero ERROR nodes

same four gates built GHC's Core. nothing to fork, nothing to harvest

Questions?

github.com/crtschin/tree-sitter-haskell-contrib

grammars for `.cabal`, `cabal.project`, and GHC's Core / STG / Cmm dumps.